

---

THE EVOLUTION OF MACHINE LEARNING AND AI · WEEK 3 OF 12

# How Machines Learn

*One idea, four pictures, and the code explained three times*

---

Live lecture · 2 hours · no code is run in class · everything marked ★ is quiz material

# How to use this lecture

## ★ Red boxes = quiz material

If it is in a red box, it can be asked in the Week 6 quiz. If it is not in a red box, it cannot. There are eight red boxes today.

## Grey boxes = extra

Optional. For people who are curious. Skip them freely. They point to the full booklet, which you are not required to read.

## IN PLAIN ENGLISH

You need no maths beyond **add, subtract, multiply**. There are two formulas in the whole lecture, and I will say each one in words first.

**Your time budget this week:** this lecture (2 h) · the lab (about 2 h, six exercises, one per part of today) · the **core** booklet (18 pages, about 45 min). That is all. Nothing else is required.

## A machine learns by guessing, checking, and nudging its numbers, thousands of times.



Every model we meet this term, from a straight line to a chatbot, runs this same four-step loop. Learn the loop today and the rest of the module is “a different first box”.

# Today's route

---

|               |                                                                                             |                                                                                            |
|---------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Part 1</b> | What “learning” means. A model is a formula with dials.                                     |  15 min |
| <b>Part 2</b> | Four shapes of model. <b>The code, explained 1: seven models, one table.</b>                |  23 min |
| <b>Part 3</b> | Memorising is not learning. How to spot it.                                                 |  10 min |
| <b>Part 4</b> | Walking downhill. The step size. <b>The code, explained 2: the walker in six lines.</b>     |  22 min |
| <b>Part 5</b> | One neuron, its wall, and the hidden layer. <b>The code, explained 3: the tiny network.</b> |  20 min |
| <b>Part 6</b> | Blame, and the five lines of PyTorch you will copy all term.                                |  13 min |
| <b>Wrap</b>   | Your week, in order. What the quiz can ask.                                                 |  7 min  |

# The one picture: a hillwalker in fog

## EVERYDAY ANALOGY

You are on a hillside in thick fog, trying to get to the bottom of the valley. You cannot see the valley. You cannot see ten metres. **But you can feel the slope under your feet.** So you take a step downhill. Then you feel again, and take another.

That is the whole of machine learning. **Height** = how wrong the model is. **Your position** = the model's numbers. **A step** = one small change to those numbers. The valley floor is where the model is least wrong.

## KEY IDEA

Every new word today is just a fact about this walk. When you feel lost, come back to the walker.

## PART 1

# What “learning” means

*A model is a formula with dials. Learning is turning the dials.*



# A model is a formula with dials

---

Guess a house price from its size:

$$\text{price} = a \times \text{size} + b$$

**a** and **b** are the dials. Set them badly and every guess is wrong. Set them well and the guesses are close. The model is “how the formula looks”; the **dials** are what gets learned.

## IN PLAIN ENGLISH

**Learning = turning the dials until the guesses are close to the real answers.** The computer does the turning. That is it. Everything else today is detail about how it turns them.

# The loop, in four words



Then go round again. Thousands of times. **Every model from now to Week 12 is this loop** with a different formula in the first box.

# Two words you will hear every week

## Parameters

The dials the **computer** turns. The **a** and **b** in the house formula. A big language model has billions of them; today's tiny network has nine.

## Hyperparameters

The settings **you** choose before training starts. How big a step to take. How many layers. How deep a tree. The computer never chooses these.

### IN PLAIN ENGLISH

Parameters: the machine's job. Hyperparameters: your job. In the lab you will change exactly **one** hyperparameter (the step size) and watch it make or break the whole thing.

# Three choices you make before pressing “train”

## CHOICE 1

**What shape of answer?** A straight line? A set of yes/no questions? A wiggly curve? (Part 2 is about this choice.)

## CHOICE 2

**How do we score a wrong answer?** Usually “how far off was the guess, squared”. This is the **loss**. (You will never have to invent one; you pick from a menu.)

## CHOICE 3

**How do we search for good dial settings?** Almost always: walk downhill, one small step at a time. (Part 4 is about this choice.)

When a textbook says “we minimise a loss function over a hypothesis class using an optimiser”, it means exactly these three choices. Same idea, longer words.

# Part 1, done

## ★ REMEMBER FOR THE WEEK 6 QUIZ

- A model is a formula with **dials** (parameters). Learning = turning the dials so guesses match answers.
- The training loop has four steps: **Guess** → **Check** → **Blame** → **Nudge**, repeated many times. (Technical names: forward, loss, backward, update.)
- **Parameters** are learned by the computer. **Hyperparameters** are chosen by you.
- Before training you make three choices: the **shape** of the answer, how to **score** a mistake, how to **search**.

EXTRA · optional · NOT on the quiz · see the full booklet if curious

The full booklet, Chapters 1–3, gives the formal versions of these statements and the notation  $(\theta, L, \hat{y})$ . Read them only if you like symbols.

## PART 2

# Four shapes of model

*Same data. Four different drawings. The model decides the shape.*

YOU ARE HERE



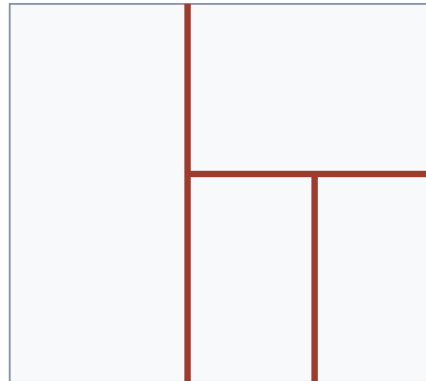
# Same dots, four ways to draw the line between them

Linear



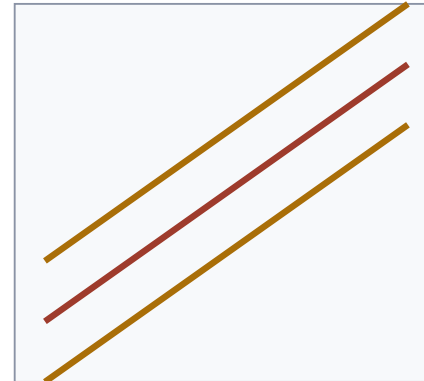
one straight line

Tree



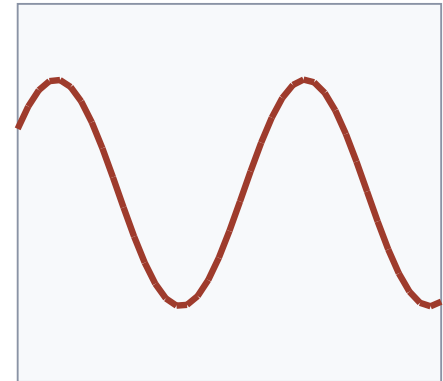
splits at right angles, one feature at a time

SVM



the line placed to maximise the margin

Neural net



simple bends, composed until it fits

A **neural network** glues together lots of small bends until the curve fits. Most flexible, hardest to read.

# Shape 1 · the straight line

---

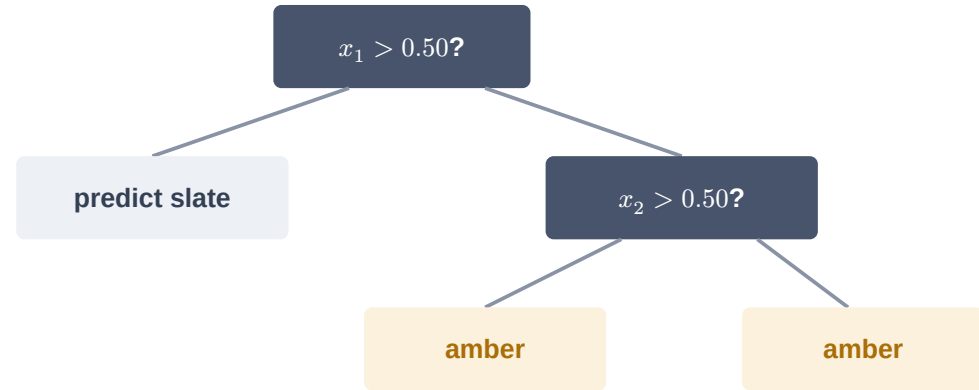
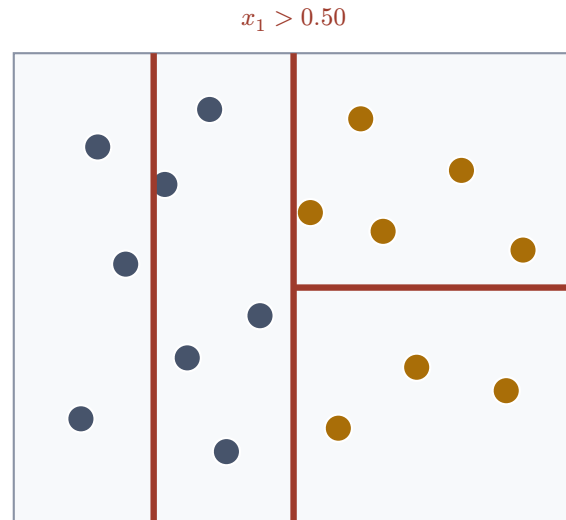
**Linear regression** predicts a **number** with a straight line. Our house formula was one:  $\text{price} = a \times \text{size} + b$ .

**Logistic regression** is the same straight line, but the answer is squashed to sit between 0 and 1 so you can read it as a **probability**: “70% chance this customer leaves”.

## IN PLAIN ENGLISH

Same line, two jobs. One predicts a number, the other a yes/no. It is still the **right first thing to try** on most real problems: fast, stable, and you can read every dial as a sentence (“each extra square metre adds €2,000”).

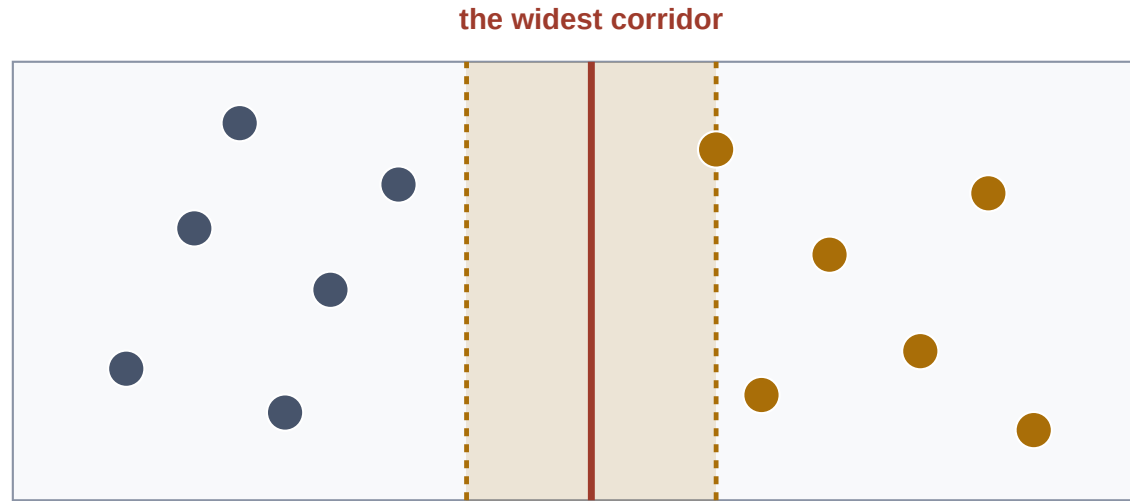
## Shape 2 · the decision tree



Every cut is **parallel to an axis**, because the tree can only ask about one feature at a time. A diagonal boundary costs it a staircase.

Keep going and eventually every box holds one dot. That is the tree **memorising**, which is Part 3's whole topic. Trees are wonderful because you can read the rules aloud; the price is right-angled boundaries and a habit of memorising.

## Shape 3 · the widest corridor



That is all you need. If no straight line works, the SVM has a trick for bending the page so that one does; that is the “kernel trick”, and it is **extra** (booklet Ch. 7).

## Shape 4 · the neural network

---

A neural network is a stack of very simple units. Each one does: **multiply, add, then bend**. Stack enough of them and you can draw almost any curve.

Most flexible of the four. Needs the most data. Hardest to explain afterwards (“why did it say no?” has no short answer).

### IN PLAIN ENGLISH

Nothing magic here. It is the house formula, repeated a few hundred times, with a bend after each repeat. Part 5 shows you the smallest possible one, nine dials, doing something the straight line cannot.

# The code, explained 1 · seven models, one table

classical.py, the heart of it

python

```
# the dots, split in two: studied / hidden
Xtr, Xte, ytr, yte = load()

for name, model in models().items():
    model.fit(Xtr, ytr)          # LEARN: turn the dials
    train = model.score(Xtr, ytr) # dots it studied
    test  = model.score(Xte, yte) # dots it never saw
    print(name, train, test, train - test) # gap
```

what it prints when you run it

| model                     | train | test  | gap   |          |
|---------------------------|-------|-------|-------|----------|
| Logistic regression       | 0.879 | 0.852 | 0.027 |          |
| Decision tree (depth 3)   | 0.915 | 0.876 | 0.039 |          |
| Decision tree (unlimited) | 1.000 | 0.857 | 0.143 | <- memo. |
| Random forest             | 1.000 | 0.890 | 0.110 |          |
| SVM (linear)              | 0.885 | 0.843 | 0.042 |          |
| SVM (RBF kernel)          | 0.915 | 0.895 | 0.020 | <- best  |
| Neural net (8, 8)         | 0.887 | 0.848 | 0.040 |          |

## WHAT EACH LINE DOES

`load()` makes the dots and hides a third of them in a drawer (the **test** set). `fit` is the whole training loop in one word: the computer turns the dials. `score` counts how many dots it gets right. We score twice, on the dots it studied and on the hidden ones, and print the difference. That last column is Part 3.

## IN THE LAB

**Exercise 2** runs this and shows the seven boundaries; **Exercise 3** lets the tree keep asking questions and watches the gap open.

## Part 2, done

### ★ REMEMBER FOR THE WEEK 6 QUIZ

- Four shapes of model: **straight line** (linear / logistic regression), **right-angled steps** (decision tree), **widest corridor** (SVM), **many small bends** (neural network).
- **Choosing a model is choosing an assumption** about the shape of the answer. Week 2 called this the good kind of “bias”.
- Trees are readable but memorise. Lines are simple and stable. Networks are flexible but need data and are hard to explain.

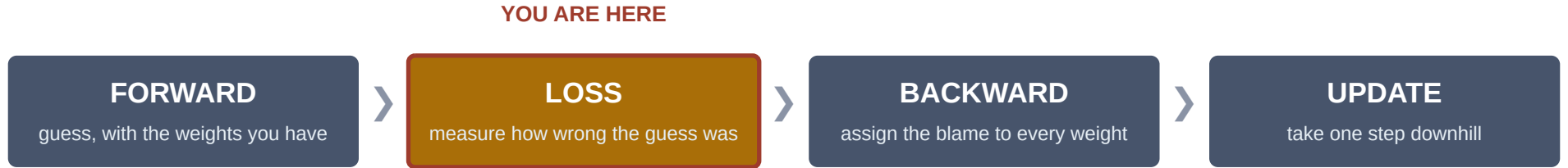
EXTRA · optional · NOT on the quiz · see the full booklet if curious

Random forests (average many trees), soft margins, the kernel trick and the maths of each model are in the full booklet, Chapters 4–7. Not needed for the quiz.

## PART 3

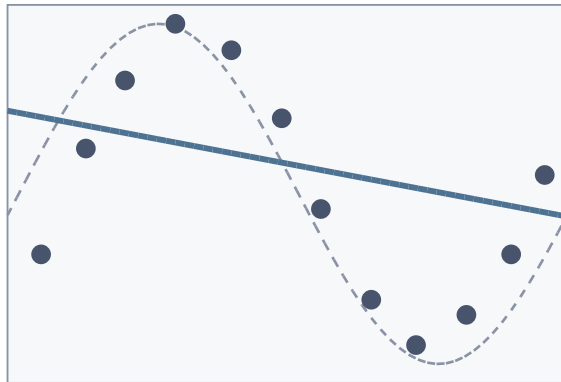
# Memorising is not learning

*The most useful ten minutes in the module*



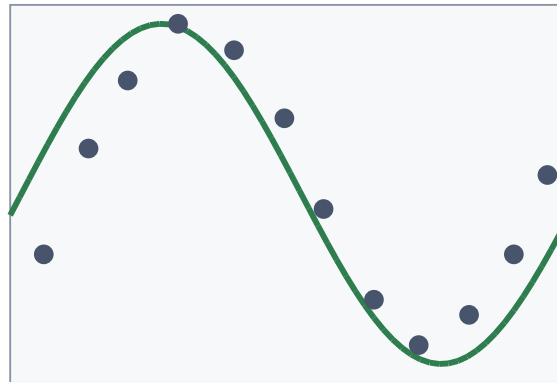
# The same twelve dots, three times

UNDERFIT



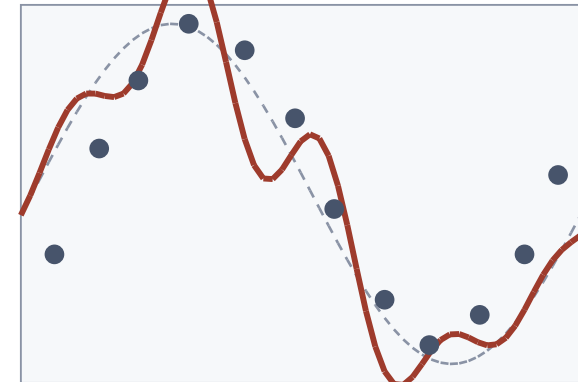
train 0.204 · true 0.228

about right



train 0.033 · true 0.021

OVERFIT



train 0.012 · true 1499.8

**Memorised.** Passes through every dot exactly. Best score on the dots it studied, and **hopeless** on new dots (true error 1500 instead of 0.02). This is **overfitting**. It learned the noise, not the pattern.

## How would you know? Look at the gap.

the two rows that matter from the table on slide 18

|                           | model | train | test  | gap   |
|---------------------------|-------|-------|-------|-------|
| Decision tree (depth 3)   |       | 0.915 | 0.876 | 0.039 |
| Decision tree (unlimited) |       | 1.000 | 0.857 | 0.143 |

The unlimited tree got **every single training dot right**, and is beaten on new dots by a tree a fraction of its size. A perfect training score is **not** good news. It is the symptom.

### IN PLAIN ENGLISH

**Always look at the gap between the training score and the test score.** Small gap: the model learned the pattern. Big gap: it memorised the examples. Student who memorises the past paper, meets a new question.

## Part 3, done

### ★ REMEMBER FOR THE WEEK 6 QUIZ

- **Underfitting**: the model is too simple to capture the pattern. Bad on training data **and** new data.
- **Overfitting**: the model memorised the training data, noise and all. Great on training data, bad on new data.
- You spot overfitting by the **gap** between training score and test score. Never report training accuracy on its own.
- Test data must be data the model has **never seen**. That is the whole point of it.

**At home: double-click [week3-lab/interactive/gradient-descent-explorer.html](#). No install. It is the next part, with a slider.**

EXTRA · optional · NOT on the quiz · see the full booklet if curious

The bias–variance dartboard and why random forests memorise yet still generalise: full booklet, Chapter 8.

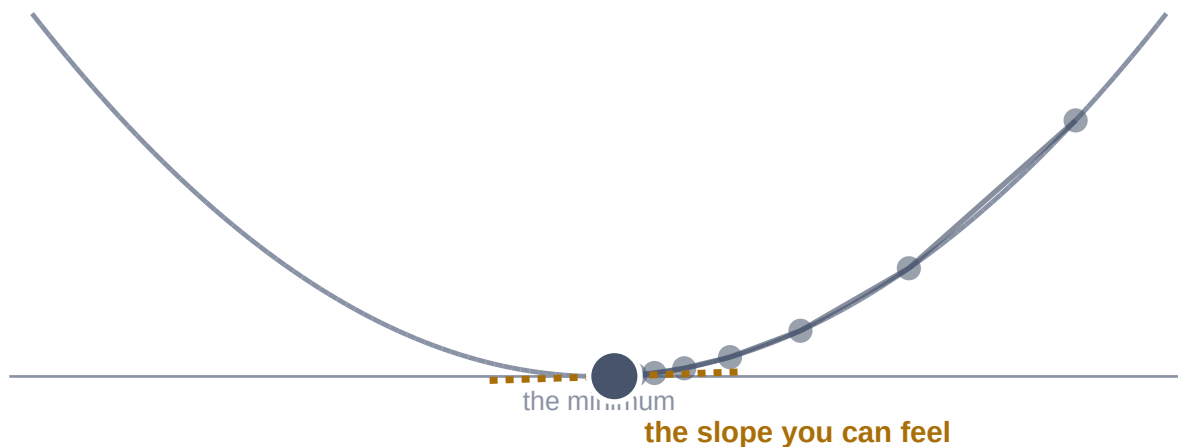
## PART 4

# Walking downhill

*How the dials actually get turned · and the one number that ruins it*



# Feel the slope, step down, repeat



## STEP 8

|      |       |
|------|-------|
| $x$  | 0.134 |
| grad | 0.134 |
| loss | 0.009 |

Notice the steps getting **smaller** on their own. Nobody told it to slow down: the gradient shrinks as it arrives.

Eight steps, each shorter than the last, and **nobody told it to slow down**. Near the bottom the slope is gentle, so the steps shrink on their own. That is gradient descent. That is the whole method.

# The one formula of the day

---

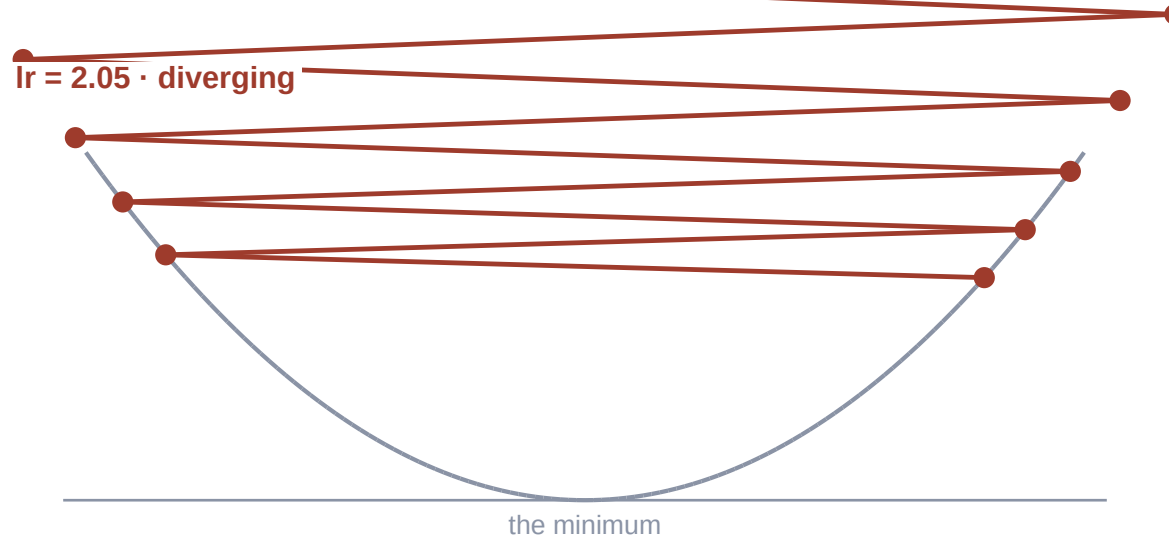
$$\text{new dial} = \text{old dial} - (\text{step size}) \times (\text{slope})$$

**Slope** is the computer's word "gradient": how steeply the loss changes if you turn this dial a little. **Step size** is the **learning rate**. The minus sign means "go **down**, not up".

$$\theta \leftarrow \theta - \eta \cdot \frac{\partial L}{\partial \theta}$$

the same sentence with symbols.  $\theta$  = the dial,  $\eta$  = step size,  $\partial L / \partial \theta$  = slope. You will see this written in books; now you can read it.

# The step size is the whole ballgame



**Far too big (2.05).** Each bounce is bigger than the last. The loss shoots off to infinity and the program prints nan. **No error message. It just breaks.**

# The code, explained 2 · the walker, in six lines

gradient\_descent.py, the whole method

python

```
def descend(f, df, x0, lr, steps):
    x = x0 # where the walker starts
    for step in range(steps): # repeat
        slope = df(x) # feel the slope
        x = x - lr * slope # dial - step size x slope
    return x

for lr in (0.02, 0.1, 0.9, 1.98, 2.05):
    descend(bowl, bowl_slope, 4.0, lr, 40)
```

what it prints when you run it

| lr   | final_loss | verdict                           |
|------|------------|-----------------------------------|
| 0.02 | 1.5892     | crawling: still going at 40 steps |
| 0.10 | 0.0017     | converged smoothly                |
| 0.90 | 0.0000     | converged smoothly                |
| 1.98 | 1.5892     | oscillating: side to side         |
| 2.05 | 396.4915   | DIVERGING: bigger every bounce    |

## WHAT EACH LINE DOES

Six lines are the entire method.  $df(x)$  is the slope at the current position. The next line is the one formula of the day, written in code. The loop runs it forty times. Then we call it five times with five step sizes, and only the step size changes the story.

## IN THE LAB

**Exercise 4** prints the walk one step per row, then these five step sizes, then lets you try your own and find the cliff. The browser toy `gradient-descent-explorer.html` is the same thing with a slider.

## Not every hill is a bowl

```
python gradient_descent.py · three starting points, same step size

start  final_loss
-2.5   6.6564
 1.0   6.6564
 5.5   8.9482  <- ended up in a different, worse dip
```

On a bumpy hill, the walker can settle in a **small dip** that is not the valley floor. It feels flat under its feet, so it stops. This is called a **local minimum**. Where you **start** can decide where you **end**.

EXTRA · optional · NOT on the quiz · see the full booklet if curious

In practice, with millions of dials, this matters less than the picture suggests, and there are fixes (momentum, restarts, noise). Full booklet, Chapter 10. Not on the quiz.

## Part 4, done

### ★ REMEMBER FOR THE WEEK 6 QUIZ

- **Gradient descent**: feel the slope, take a small step downhill, repeat. The steps shrink on their own near the bottom.
- The formula: **new dial = old dial – step size × slope**.
- The step size is the **learning rate**. Too small: crawls. Too big: bounces. Far too big: explodes to nan with no error message.
- A **local minimum** is a small dip that is not the true bottom; the walker can get stuck there.

EXTRA · optional · NOT on the quiz · see the full booklet if curious

Batch vs mini-batch vs stochastic descent, momentum and saddle points: full booklet Chapters 9–10.

## PART 5

# One neuron, its wall, and the hidden layer

*Week 1's promise, kept, with four dots*

YOU ARE HERE



## One artificial neuron (1958)

Take the inputs. Multiply each by a dial (a **weight**). Add them up. If the total is above a threshold, say **yes**; otherwise say **no**.

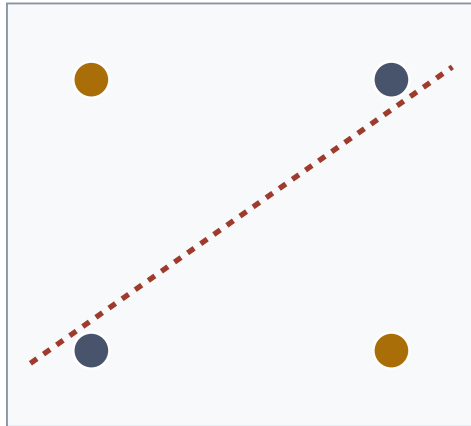
$$\hat{y} = \begin{cases} 1 & \text{if } w_1x_1 + w_2x_2 + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

that is the entire model. Rosenblatt called it the **perceptron**. It is the house formula with a yes/no bolted on.

It learns by the loop: guess, check, and if wrong, **nudge the weights toward the right answer**. Rosenblatt proved it will always succeed... **if** the two colours can be separated by a straight line.

# The wall: XOR (1969)

Input space

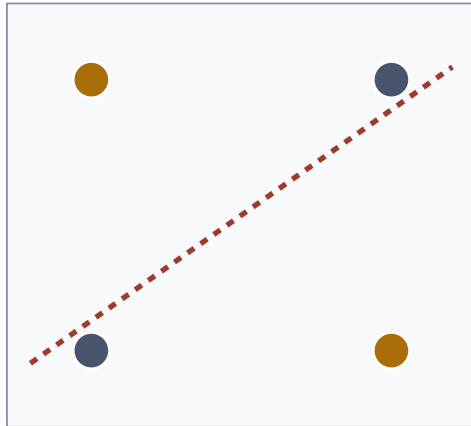


no line separates them

Try to draw **one straight line** with both amber dots on one side and both slate dots on the other. You cannot. Nobody can. There is no such line, so the perceptron can never learn this. That is the wall Week 1 talked about.

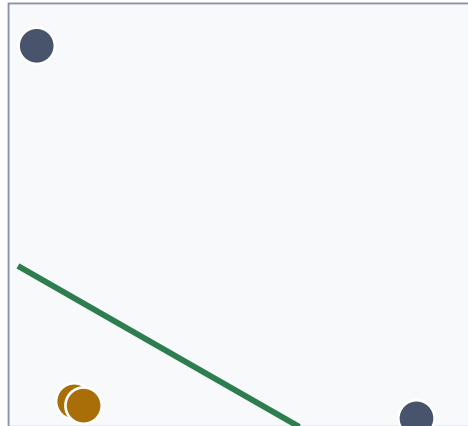
# The way through: a hidden layer moves the dots

Input space



no line separates them

After one hidden layer



now one line does

**A hidden layer moves the dots so that a straight line works.** That is the whole idea, and it is what took the field seventeen years to learn how to train.

## Why each neuron needs a “bend”

---

A straight line, followed by another straight line, is still... a straight line. Stack a hundred straight-line layers and you still cannot draw XOR.

So after each neuron adds things up, we put the total through a **bend**: a squashing function. The classic one (the **sigmoid**) squeezes any number into 0 to 1. The modern one (**ReLU**) just replaces negatives with zero.

### IN PLAIN ENGLISH

No bend, no curves. With the bend, a network of tiny straight pieces can draw anything. In the Playground, switch the bend **off** and watch XOR fail again.

# The code, explained 3 · one hidden layer, forward and back

scratch\_net.py, the tiny network

python

```
def forward(self, X):
    # hidden: multiply, add, BEND
    self.h = sigmoid(X @ self.W1 + self.b1)
    # output: same again
    return sigmoid(self.h @ self.W2 + self.b2)

def backward(self, X, Y, lr):
    out = self.forward(X)
    # blame at the output, then passed back
    b_out = (out - Y) * out * (1 - out)
    b_h = b_out @ self.W2.T * self.h * (1 - self.h)
    # NUDGE every dial
    self.W2 -= lr * self.h.T @ b_out
    self.W1 -= lr * X.T @ b_h
```

what it prints when you run it

```
perceptron, no hidden layer
-> 2 of 4 right. Stuck. (1969)
2 hidden neurons, no bend
-> 2 of 4. Outputs 0.5 for all.
2 hidden neurons, sigmoid
-> 4 of 4. Loss 0.26 -> 0.0025 (1986)

hidden layer moved
(0,1) to (0.10, 0.03)
(1,0) to (0.10, 0.02)
on top of each other
```

## WHAT EACH LINE DOES

forward: multiply, add, bend, twice (@ is multiply-and-add, sigmoid is the bend). backward: blame at the output, passed back through w2, then every dial nudged by step size times its blame. The loop in ten lines.

## IN THE LAB

**Exercise 5** runs the perceptron (fails), this network (succeeds, prints where the dots moved), and the same network with the bend removed (fails again).

## Part 5, done

### ★ REMEMBER FOR THE WEEK 6 QUIZ

- A **perceptron** (1958) is one neuron: multiply inputs by weights, add, then yes/no. It can only draw a straight line.
- **XOR** (1969): four dots that no straight line can separate. The perceptron can never learn it.
- A **hidden layer** moves the dots into a new arrangement where a straight line **does** work.
- Each neuron needs a **bend** (activation function, e.g. sigmoid or ReLU). Without it, many layers collapse into one straight line.

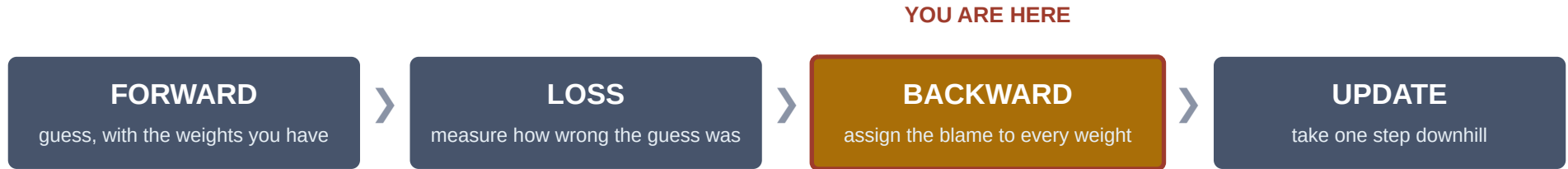
EXTRA · optional · NOT on the quiz · see the full booklet if curious

Rosenblatt's convergence theorem, the vanishing-gradient problem and why ReLU won: full booklet Chapters 11 and 14.

## PART 6

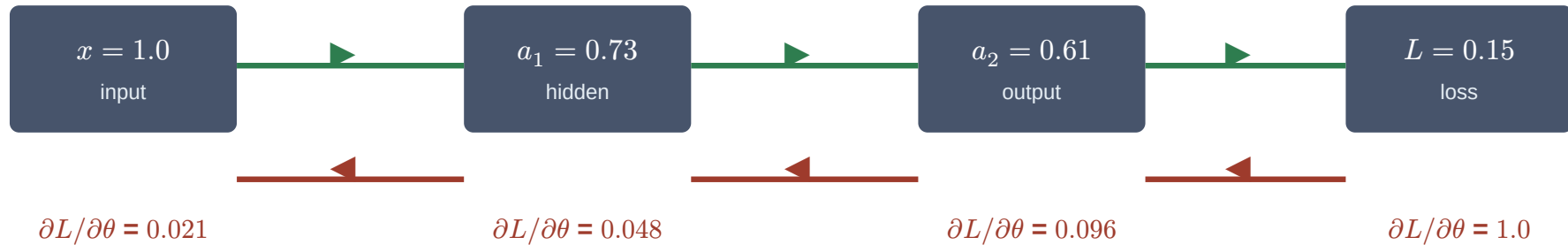
# Blame, and the five lines

*The hard step, in one sentence · then the code you will copy all term*



# Blame, going backwards

**FORWARD · keep every value on the way through**



**BACKWARD · multiply the local derivatives, right to left**

The gradient **shrinks** at every layer: 0.096 to 0.048 to 0.021. Multiply that by ten layers of sigmoid and the first layer receives essentially nothing.

...then the hidden layer gets its share, then the input weights get theirs. **That is backpropagation:** passing blame backwards, layer by layer, so every dial knows which way to turn. It is the “Blame” box of the loop.

## You do not have to do the blame by hand

---

Working out the blame is the **chain rule** from calculus, applied over and over. In the lab, `scratch_net.py` writes it out in about ten lines so you can **see** it happen. You do not need to be able to write it yourself.

In real life, PyTorch does it in **one line**: `loss.backward()`. You write the forward pass; it records every step and walks back through them for you.

### IN PLAIN ENGLISH

For the quiz you need one sentence: **backpropagation is the method for working out how much each weight is to blame for the loss, starting from the output and working backwards.** Gradient descent then uses that to nudge the weights.

## Why did this wait until 2012?

Backpropagation was published in **1986**. Nothing important changed in the maths for the next 26 years. So why did deep learning explode in 2012?

### Data · 2009

ImageNet: 14 million labelled pictures. The loop needs examples to learn from.

### Compute · 2007

Graphics cards (GPUs) turn out to be perfect for millions of small multiply-and-adds.

### Small fixes

A better bend (ReLU), better starting values for the dials, a few tricks.

### KEY IDEA

AlexNet (2012) used **1986's algorithm** on **2009's data** with **2007's hardware**. The recipe was old. The ingredients arrived late.

# The five lines you will copy all term

week3-lab/torch\_intro.py

python

```
for epoch in range(epochs):  
    optimiser.zero_grad()      # 0. wipe last time's blame  
    pred = model(X)           # 1. GUESS    (forward)  
    loss = loss_fn(pred, Y)    # 2. CHECK    (loss)  
    loss.backward()           # 3. BLAME    (backward)  
    optimiser.step()          # 4. NUDGE    (update)
```

Lines 1 to 4 **are** the four boxes from the start of the lecture. Line 0 is housekeeping: PyTorch remembers last time's blame unless you wipe it. Forget line 0 and it **still runs**, just wrong.

## IN PLAIN ENGLISH

You will paste these five lines into every lab from here to Week 12. Nothing in them is magic: you have now seen what each one does by hand. PyTorch is **optional** this week; the lab works without it.

## Part 6, done

### ★ REMEMBER FOR THE WEEK 6 QUIZ

- **Backpropagation** works out how much each weight is to blame for the loss, starting at the output and working backwards. It is the “Blame” step. Gradient descent then does the nudging.
- Deep learning took off in **2012** because **data** (ImageNet) and **compute** (GPUs) finally arrived for a 1986 algorithm.
- PyTorch’s training loop is five lines: **zero\_grad, forward, loss, backward, step**. They map onto the loop.
- `loss.backward()` replaces the hand-written blame calculation. It computes gradients; it does not “learn”.

EXTRA · optional · NOT on the quiz · see the full booklet if curious

The chain rule written out, the gradient check, and the `zero_grad` trap in detail: full booklet Chapters 15 and 16.

ABOUT FIVE HOURS TOTAL · DO THEM IN THIS ORDER · STOP WHEN THE LIST ENDS

## Your week, in order

- 1 **This lecture.** You are nearly done.  2 h
- 2 **Core booklet** (18 pages). Same six parts as today, with the pictures. Read once, slowly.  45 min
- 3 **The lab: six exercises, one per part of today, in this order.** Each does on your machine what the slides did on the screen: the dials, the four shapes, the gap, the walk, XOR, blame by hand.  6 × 20 min
- 4 **Check** SUBMISSION.md against your run (it is already filled in), add your name, upload it.  10 min
- + **Optional, only if you want more:** the five “break it on purpose” challenges, PyTorch, and the full booklet.  ∞

Every exercise names the slides it follows, tells you **what to run**, **what you should see**, and **where to look** for each answer. If you are spending more than 40 minutes on an exercise, stop and post in the channel: that is a bug in the material, not in you.

## What the Week 6 quiz can ask about Week 3

---

- ★1 The four steps of the training loop, in order, and what each does.
- ★2 Parameters vs hyperparameters. The three choices before training.
- ★3 The four model shapes and what each assumes about the answer.
- ★4 Underfitting vs overfitting, and how the train–test **gap** reveals it.
- ★5 Gradient descent in words; the one formula; what the learning rate does when too small / too big.
- ★6 What a local minimum is.
- ★7 The perceptron, why it fails on XOR, and what a hidden layer does about it. Why the “bend” is needed.
- ★8 Backpropagation in one sentence. Why 2012. The five PyTorch lines and which loop step each is.

The core booklet ends with 20 practice questions **with answers**, one page. Every question is one of these eight in disguise.



**Guess. Check. Blame. Nudge. Everything else is a different first box.**

Questions now, or in the channel any time this week. Next week: why the **shape** of a network is itself an assumption about the world.